After we multiply the query and key together for object *a*, we get an unnormalized attention weight, $w_{a,a} = \langle a_Q, a_K \rangle$, which is a single scalar value for the self-attention between *a* and *a* (itself). We then do the same for the pairwise interaction between *a* and *b*, and *b* and *a*, and *b* and *b*, so we get a total of four attention weights. These could be arbitrarily small or large numbers, so we normalize all the attention weights using the softmax function, which as you may recall, takes a bunch of numbers (or a vector) and normalizes all the values to be in the interval [0,1] and forces them to sum to 1 so that they form a proper discrete probability distribution. This normalization forces the attention mechanism to only attend to what is absolutely necessary for the task. Without this normalization, the model could easily attend to everything, and it would remain un-interpretable.

Once we have the normalized attention weights, we can collect them into an attention weight matrix. In our simple example with two objects, *a* and *b*, this would be a 2 × 2 matrix. We can then multiply the attention matrix by each value vector, which will increase or decrease the elements in each vector value according to the attention weights. This will give us a set of new and updated node vectors. Each node has been updated based on the strength of its relationships to other nodes.

Rather than multiplying individual vectors together one at a time, we can instead multiply entire node matrices together. Indeed, we can efficiently combine the three steps of key-query multiplication (to form an attention matrix), then attention-matrix with value-matrix multiplication, and finally normalization, into an efficient matrix multiplication,

$$\hat{N} = softmax(QK^T)V$$

where $Q: \mathbb{R}^{n \times f}$, $K^T: \mathbb{R}^{f \times n}$, $V: \mathbb{R}^{n \times f}$, where *n* is the number of nodes, *f* is the dimension of the node feature vector, *Q* is the query matrix, *K* is the key matrix, and *V* is the value matrix.

You can see that the result of QK^T will be a $n \times n$ dimensional matrix, which is an adjacency matrix as we described earlier, but in this context we call it the attention (weight) matrix. Each row and column represent a node.